

Actividad Unidad 4

Cliente RESTful Web

Desarrollo de Web - Ingenieria del software

Campo	Descripcion
IES	Universidad de Cartagena
Tutor	John Carlos Arrieta Arrieta
Estudiante	Josue David Fischer Yepes
Modalidad	Individual
Framework	React + TypeScript + Vite
Fecha	10/06/2026

Repositorio backend base: <https://github.com/arrietajohn/users-management-spring-boot-hexagonal-ddd>

Introduccion

El presente informe documenta el desarrollo de una aplicacion web frontend que funciona como cliente RESTful para el backend de gestion de usuarios construido con Spring Boot, arquitectura hexagonal y enfoque DDD. La solucion permite listar, consultar, crear, actualizar y eliminar registros desde una interfaz grafica, incorporando validaciones en el formulario, manejo de respuestas HTTP y mensajes de exito o error.

Objetivos

- Construir un cliente RESTful web para consumir todos los endpoints expuestos por el backend.
- Implementar operaciones CRUD completas desde una interfaz grafica clara.
- Separar el codigo en componentes, servicios, modelos, utilidades de validacion y configuracion.
- Manejar respuestas del servidor, errores HTTP y validacion previa al envio de formularios.
- Documentar la integracion, estructura del proyecto y evidencias principales de la ejecucion.

Framework seleccionado y justificacion

Se selecciono React con TypeScript y Vite. React permite organizar la interfaz en componentes reutilizables; TypeScript agrega tipado para los modelos y payloads del API; y Vite facilita un entorno de desarrollo rapido con proxy local hacia el backend. Esta combinacion cumple con los frameworks permitidos por la actividad y permite demostrar el consumo REST de forma clara.

Descripcion general del backend consumido

El backend base es una aplicacion Spring Boot organizada con arquitectura hexagonal. La capa REST expone el recurso de usuarios bajo la ruta `/api/users`. Internamente el proyecto define casos de uso para crear, listar, consultar, actualizar y eliminar usuarios, y persiste la informacion en MySQL mediante la base de datos `crud_usuarios`.

- Puerto del backend: `http://localhost:8080`.
- URL base usada por el frontend: `/api`, redirigida por Vite hacia `http://localhost:8080`.
- Recurso principal: usuarios.
- Roles validos: ADMIN, MEMBER, REVIEWER.
- Estados validos: ACTIVE, INACTIVE, PENDING, BLOCKED.

Tabla de endpoints consumidos

Operacion	Metodo	Endpoint	Datos enviados	Respuesta esperada
Listar usuarios	GET	<code>/api/users</code>	Sin cuerpo	200 OK, arreglo de usuarios
Consultar usuario	GET	<code>/api/users/{id}</code>	ID en la ruta	200 OK usuario / 404 error
Crear usuario	POST	<code>/api/users</code>	id, name, email, password, role	201 Created, usuario creado

Actualizar usuario	PUT	/api/users/{id}	name, email, password, role, status	200 OK, usuario actualizado
Eliminar usuario	DELETE	/api/users/{id}	ID en la ruta	204 No Content

Estructura del proyecto frontend

La aplicacion fue organizada siguiendo buenas practicas de React: la vista principal orquesta el estado de la pantalla, los componentes se encargan de la presentacion, el servicio HTTP centraliza el consumo del API y las utilidades encapsulan validaciones y transformaciones.

Archivo	Responsabilidad
src/models/user.ts	Interfaces User, payloads de creacion/actualizacion, roles y estados.
src/services/usersService.ts	Servicio REST con fetch, manejo de errores y metodos CRUD.
src/components/UserForm.tsx	Formulario reutilizable para crear y editar usuarios.
src/components/UsersTable.tsx	Tabla de registros con acciones de editar y eliminar.
src/components/ConfirmDialog.tsx	Confirmacion antes de eliminar registros.
src/components/StatusMessage.tsx	Mensajes de exito, informacion y error.
src/utils/userValidation.ts	Validaciones de ID, nombre, correo, contrasena, rol y estado.
vite.config.ts	Proxy /api hacia http://localhost:8080 para integracion local.

Servicios REST implementados

El archivo usersService.ts define un cliente HTTP reutilizable. Cada metodo corresponde a una operacion del API: getAll, getByid, create, update y remove. La funcion request centraliza cabeceras JSON, parseo de respuestas, manejo del codigo 204 y conversion de errores del backend al tipo ApiClientError para mostrarlos en la interfaz.

Servicio HTTP userService.ts

```
1 import type { ApiErrorResponse, CreateUserPayload, UpdateUserPayload, User } from '../models/user'
2
3 export const API_BASE_URL = (import.meta.env.VITE_API_BASE_URL ?? '/api').replace(/\/$/, '')
4
5 const USERS_ENDPOINT = `${API_BASE_URL}/users`
6
7 export class ApiClientError extends Error {
8   readonly status: number
9
10  constructor(status: number, message: string) {
11    super(message)
12    this.name = 'ApiClientError'
13    this.status = status
14  }
15 }
16
17 async function request<T>(input: RequestInfo | URL, init?: RequestInit): Promise<T> {
18   let response: Response
19
20   try {
21     response = await fetch(input, {
22       headers: {
23         'Content-Type': 'application/json',
24         ...init?.headers,
25       },
26       ...init,
27     })
28   } catch {
29     throw new Error('No hay conexion con el backend. Verifica que Spring Boot este ejecutandose.')
30   }
31
32   if (!response.ok) {
33     throw await buildApiError(response)
34   }
35
36   if (response.status === 204) {
37     return undefined as T
38   }
39
40   return response.json() as Promise<T>
41 }
42
43 async function buildApiError(response: Response) {
44   if (response.status === 502 || response.status === 504) {
45     return new ApiClientError(
46       response.status,
47       'Backend no disponible. Verifica que Spring Boot este ejecutandose en el puerto 8080.',
48     )
49   }
50
51   try {
52     const body = (await response.json()) as ApiErrorResponse
53     return new ApiClientError(response.status, body.message || response.statusText)
54   } catch {
55     return new ApiClientError(response.status, response.statusText || 'Error del servidor')
56   }
57 }
58
59 export const userService = {
60   getAll() {
61     return request<User[]>(USERS_ENDPOINT)
62   },
63
64   getById(id: string) {
65     return request<User>(`${USERS_ENDPOINT}/${encodeURIComponent(id)}`)
66   },
67
68   create(payload: CreateUserPayload) {
69     return request<User>(USERS_ENDPOINT, {
70       method: 'POST',
71       body: JSON.stringify(payload),
72     })
73   },
74
75   update(id: string, payload: UpdateUserPayload) {
76     return request<User>(`${USERS_ENDPOINT}/${encodeURIComponent(id)}`, {
77       method: 'PUT',
78       body: JSON.stringify(payload),
79     })
80   },
81 }
```

Figura 1. Servicio HTTP encargado de consumir el API REST.

Modelos y validaciones

Los modelos TypeScript reflejan los campos usados por el backend: id, name, email, role y status. La validación del formulario replica las restricciones principales del dominio: nombre mínimo de tres caracteres, correo válido, contraseña mínima de ocho caracteres y campos obligatorios antes de enviar peticiones al servidor.

```
Modelo de datos user.ts

1 export const USER_ROLES = ['ADMIN', 'MEMBER', 'REVIEWER'] as const
2 export const USER_STATUSES = ['ACTIVE', 'INACTIVE', 'PENDING', 'BLOCKED'] as const
3
4 export type UserRole = (typeof USER_ROLES)[number]
5 export type UserStatus = (typeof USER_STATUSES)[number]
6
7 export interface User {
8   id: string
9   name: string
10  email: string
11  role: UserRole
12  status: UserStatus
13 }
14
15 export interface CreateUserPayload {
16   id: string
17   name: string
18   email: string
19   password: string
20   role: UserRole
21 }
22
23 export interface UpdateUserPayload {
24   name: string
25   email: string
26   password?: string
27   role: UserRole
28   status: UserStatus
29 }
30
31 export interface ApiErrorResponse {
32   status: number
33   message: string
34 }
```

Figura 2. Interfaces y constantes usadas por el frontend.

```

Validaciones userValidation.ts

1 import type { UserFormState } from '../models/user'
2
3 export type UserFormErrors = Partial<Record<keyof UserFormState, string>>
4
5 const EMAIL_PATTERN = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/
6
7 export function validateUserForm(form: UserFormState, mode: 'create' | 'edit') {
8   const errors: UserFormErrors = {}
9   const id = form.id.trim()
10  const name = form.name.trim()
11  const email = form.email.trim()
12  const password = form.password.trim()
13
14  if (!id) {
15    errors.id = 'El ID es obligatorio.'
16  }
17
18  if (!name) {
19    errors.name = 'El nombre es obligatorio.'
20  } else if (name.length < 3) {
21    errors.name = 'El nombre debe tener al menos 3 caracteres.'
22  }
23
24  if (!email) {
25    errors.email = 'El correo es obligatorio.'
26  } else if (!EMAIL_PATTERN.test(email)) {
27    errors.email = 'Ingresa un correo valido.'
28  }
29
30  if (mode === 'create' && !password) {
31    errors.password = 'La contraseña es obligatoria.'
32  } else if (password && password.length < 8) {
33    errors.password = 'La contraseña debe tener al menos 8 caracteres.'
34  }
35
36  if (!form.role) {
37    errors.role = 'Selecciona un rol.'
38  }
39
40  if (mode === 'edit' && !form.status) {
41    errors.status = 'Selecciona un estado.'
42  }
43
44  return errors
45 }

```

Figura 3. Validaciones del formulario antes del envío.

Formulario e interfaz grafica

La interfaz permite navegar entre las funcionalidades de listar, consultar por ID, crear, actualizar y eliminar usuarios. El formulario cambia entre modo creacion y edicion, bloquea el ID durante la edicion, muestra errores de validacion y permite seleccionar roles y estados validos para el backend.

Formulario UserForm.tsx

```

1 import { Save } from 'lucide-react'
2 import type { ReactNode } from 'react'
3 import type { UserFormErrors } from '../utils/userValidation'
4 import { USER_ROLES, USER_STATUSES, type UserFormState } from '../models/user'
5
6 interface UserFormProps {
7   form: UserFormState
8   mode: 'create' | 'edit'
9   errors: UserFormErrors
10  isSubmitting: boolean
11  onChange: (form: UserFormState) => void
12  onSubmit: () => void
13 }
14
15 export function UserForm({
16   form,
17   mode,
18   errors,
19   isSubmitting,
20   onChange,
21   onSubmit,
22 }: UserFormProps) {
23   const updateField = <K extends keyof UserFormState>(field: K, value: UserFormState[K]) => {
24     onChange({ ...form, [field]: value })
25   }
26
27   return (
28     <form
29       className="user-form"
30       onSubmit={(event) => {
31         event.preventDefault()
32         onSubmit()
33       }}
34     >
35       <Field label="ID" error={errors.id}>
36         <input
37           value={form.id}
38           onChange={(event) => updateField('id', event.target.value)}
39           disabled={mode === 'edit'}
40           placeholder="00000000-0000-0000-0000-000000000002"
41         />
42       </Field>
43
44       <Field label="Nombre" error={errors.name}>
45         <input
46           value={form.name}
47           onChange={(event) => updateField('name', event.target.value)}
48           placeholder="Nombre completo"
49         />
50       </Field>
51
52       <Field label="Correo electronico" error={errors.email}>
53         <input
54           type="email"
55           value={form.email}
56           onChange={(event) => updateField('email', event.target.value)}
57           placeholder="usuario@example.com"
58         />
59       </Field>
60
61       <Field label={mode === 'create' ? 'Contraseña' : 'Nueva contraseña'} error={errors.password}>
62         <input
63           type="password"
64           value={form.password}
65           onChange={(event) => updateField('password', event.target.value)}
66           placeholder={mode === 'create' ? 'Mínimo 8 caracteres' : 'Opcional'}
67         />
68       </Field>
69
70       <div className="form-row">
71         <Field label="Rol" error={errors.role}>
72           <select value={form.role} onChange={(event) => updateField('role', event.target.value as UserFormState['role'])}>
73             {USER_ROLES.map((role) => (
74               <option value={role} key={role}>
75                 {role}
76               </option>
77             ))}
78           </select>
79         </Field>
80
81         <Field label="Estado" error={errors.status}>
82           <select
83             value={form.status}
84             onChange={(event) => updateField('status', event.target.value as UserFormState['status'])}
85             disabled={mode === 'create'}

```

Figura 4. Componente de formulario para crear y editar usuarios.

Captura de ejecucion de la aplicacion

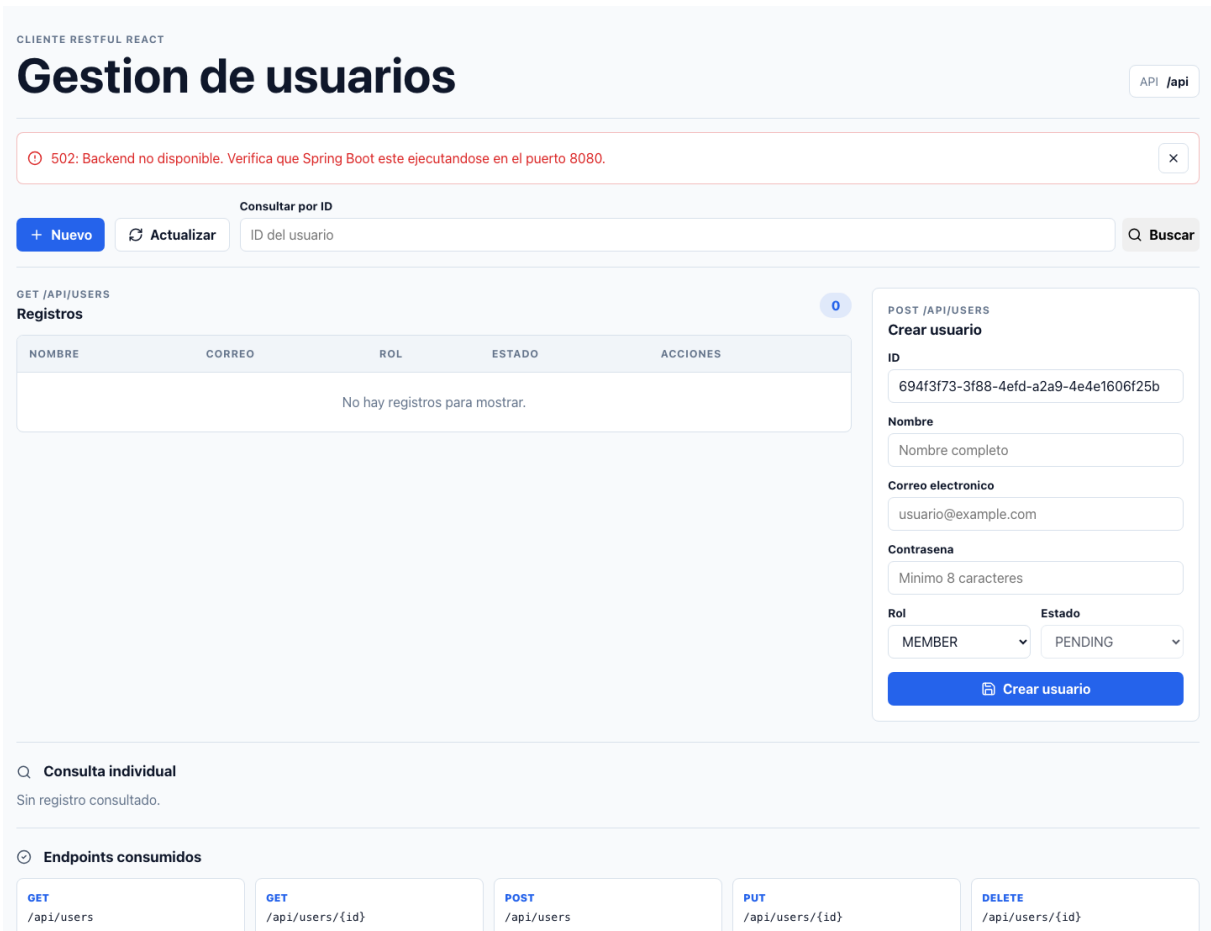


Figura 5. Interfaz principal del cliente RESTful en ejecucion local.

La captura evidencia la pantalla principal con acciones de nuevo registro, actualizacion del listado, consulta por ID, tabla de usuarios, formulario y resumen de endpoints consumidos.

Evidencia de operaciones CRUD

Operacion	Accion en la interfaz	Endpoint usado	Resultado esperado
Crear	Diligenciar formulario y pulsar Crear usuario	POST /api/users	Mensaje de exito y registro en la tabla
Consultar	Ingresar ID y pulsar Buscar	GET /api/users/{id}	Detalle del usuario consultado
Actualizar	Pulsar editar, modificar campos y guardar	PUT /api/users/{id}	Datos actualizados en tabla y detalle
Eliminar	Pulsar eliminar y confirmar	DELETE /api/users/{id}	Registro eliminado y mensaje de exito

Para registrar la evidencia final de la sustentacion se debe ejecutar el backend con MySQL y SMTP configurados, realizar las cuatro operaciones anteriores y capturar cada resultado. El frontend ya incluye las rutas, payloads y controles necesarios para ejecutar el ciclo CRUD.

Problemas encontrados y solucion aplicada

- El backend no expone configuracion CORS visible en el repositorio. Se configuro proxy de Vite para consumir /api desde el mismo origen durante desarrollo.
- Cuando el backend no esta disponible, el proxy retorna 502. El servicio transforma ese estado en un mensaje claro para el usuario.
- Los formularios pueden enviar datos invalidos si no se validan en cliente. Se implemento validacion previa para nombre, correo, contrasena, rol y estado.
- La eliminacion es una accion irreversible. Se agrego un dialogo de confirmacion antes de ejecutar DELETE.

Instrucciones de instalacion y ejecucion

Paso	Comando / accion
1	Ejecutar el backend Spring Boot en http://localhost:8080 .
2	Configurar MySQL con la base crud_usuarios usando schema.sql del backend.
3	En el frontend ejecutar npm install.
4	Ejecutar npm run dev.
5	Abrir http://localhost:5173 o la URL indicada por Vite.

Conclusiones

- La solucion implementa un cliente RESTful funcional y organizado para el recurso de usuarios del backend modelo.
- La separacion entre componentes, servicio HTTP, modelos y utilidades facilita el mantenimiento y cumple con la organizacion solicitada.
- El uso de TypeScript reduce errores al definir payloads y respuestas alineadas con el API.
- La interfaz permite demostrar las operaciones CRUD y maneja validaciones, confirmaciones y errores del servidor.

Referencias consultadas

- Repositorio backend base: <https://github.com/arrietajohn/users-management-spring-boot-hexagonal-ddd>
- Documentacion de React: <https://react.dev/>
- Documentacion de Vite: <https://vite.dev/>
- Documentacion de Spring Boot: <https://spring.io/projects/spring-boot>
- Consigna de la Actividad Unidad 4 - Desarrollo de Web.